

Spring Security 6.x: От фильтров сервлета до JWT, Refresh и OAuth 2.0 / OIDC

Глубокое академическое и прикладное руководство по проектированию систем аутентификации и авторизации в современных Java/Spring приложениях.

Целевая аудитория: Разработчики Java / Spring Boot, инженеры корпоративных систем, архитекторы.

Технологический стек: Java 17/21, Spring Boot 3.x, Spring Security 6.x, JWT 0.12+, OAuth 2.1, OpenID Connect 1.0, RFC 7519 / 6749 / 7636.

Формат материала: Теория угроз → Архитектура компонентов → Цветные блок-схемы потока вызовов → Пошаговое внедрение → Построчный разбор кода → Реальные enterprise-паттерны.

Содержание учебника

- | | |
|--|---|
| 1. Фундамент безопасности веб-приложений и модель угроз | 7. Архитектура и реализация JWT (JSON Web Tokens) |
| 2. Понятийный аппарат Spring Security | 8. Двухтокенная схема: JWT Access + Refresh Token |
| 3. Архитектура под капотом: от сокета до контроллера | 9. Протокол OAuth 2.0: Делегирование авторизации |
| 4. Факторы и методы аутентификации (2FA, TOTP, FIDO2) | 10. OpenID Connect (OIDC): Слой идентификации |
| 5. Базовая настройка Spring Security с нуля | 11. Внедрение OAuth 2.0 / OIDC в Spring Boot |
| 6. HTTP Basic Authentication: протокол и код | 12. Приложение: Промышленная архитектура безопасности |
| 13. Руководство по работе с официальной документацией Spring и техническим английским (A2+/B1) | |

Глава 1. Фундамент безопасности веб-приложений и модель угроз

Любое веб-приложение функционирует в принципиально враждебной среде. Каждый байт, поступающий по сети через сетевой сокет от внешнего клиента (веб-браузера, мобильного приложения, curl-запроса или автоматизированного скрипта), должен считаться потенциально скомпрометированным. Безопасность не может обеспечиваться «сокрытием» кнопок на клиенте или проверками на уровне пользовательского интерфейса (UI). Единственным достоверным рубежом обороны является серверное приложение.

1.1. Триада информационной безопасности (CIA Triad)

Компонент	Определение	Угроза нарушения	Механизм защиты в Spring Security
Confidentiality (Конфиденциальность)	Доступ к информации разрешен исключительно авторизованным субъектам.	Утечка паролей, чтение чужих персональных данных или финансовых записей.	Шифрование каналов связи (TLS), хеширование паролей (BCrypt/Argon2), ролевой доступ (RBAC, <code>@PreAuthorize</code>).
Integrity (Целостность)	Гарантия неизменности и достоверности данных в процессе передачи и хранения.	Подделка суммы платежа, несанкционированное изменение ролей пользователя.	Криптографические цифровые подписи (HMAC, RSA в JWT), CSRF-защита, транзакционная валидация.
Availability (Доступность)	Своевременный и беспрепятственный доступ легитимных пользователей к сервису.	DDoS-атаки, блокировка учетных записей исчерпанием пула соединений.	Ограничение частоты запросов (Rate Limiting), эффективное кэширование, защита от блокировки потоков сервлета.

1.2. Анализ ключевых веб-угроз (OWASP Top 10)

- **Broken Access Control (Нарушение контроля доступа):** Ситуация, когда пользователь с ролью обычного оператора обращается к URL `DELETE /api/v1/users/42`. Если проверка прав отсутствует или выполнена только на клиенте, операция выполнится. В Spring Security защита реализуется через декларативный контроль URL (`authorizeHttpRequests`) и метод-секьюрити (AOP-прокси на сервисах).
- **Cryptographic Failures (Сбои криптографии):** Хранение паролей в открытом виде или использование устаревших алгоритмов (MD5, SHA-1). Spring Security принудительно требует реализации интерфейса `PasswordEncoder`, исключающего хранение открытого текста.
- **Cross-Site Scripting (XSS):** Внедрение вредоносного JavaScript в страницу жертвы. XSS позволяет украсть токены из `localStorage`. Защита: экранирование вывода, политика Content Security Policy (CSP) и использование флага `HttpOnly` для cookie.

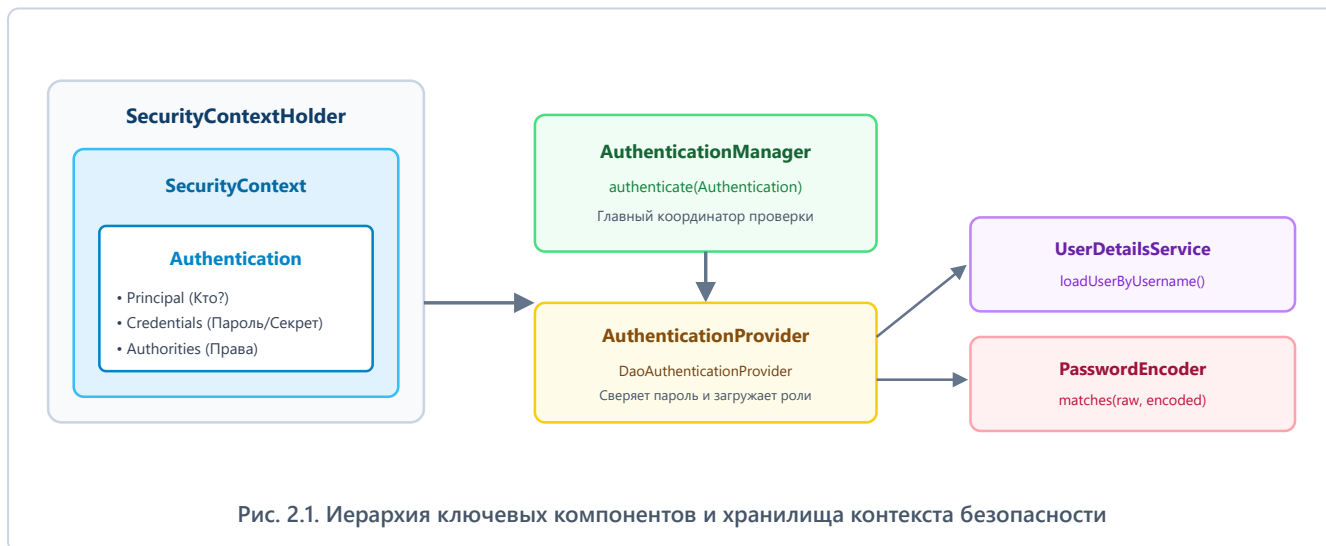
- **Cross-Site Request Forgery (CSRF):** Атака, при которой сторонний сайт отправляет скрытый запрос на защищенный ресурс от имени авторизованного пользователя, используя автоматически прикрепляемые браузером cookie сессии (`JSESSIONID`). Spring Security по умолчанию активирует `CsrfFilter` с генерацией уникального токена синхронизации (Synchronizer Token Pattern).
- **Session Hijacking & Replay Attacks:** Перехват идентификатора сессии или токена доступа. Защита: принудительный HTTPS (HSTS), короткое время жизни токенов доступа (Access Token TTL), ротация Refresh-токенов.

1.3. Базовые защитные протоколы и заголовки

- **TLS / HTTPS (Transport Layer Security):** Протокол криптографической защиты транспортного уровня. Шифрует весь HTTP-пакет (URL пути, заголовки, тело запроса, cookie). Без HTTPS передача паролей (в том числе Basic Auth) и токенов (Bearer JWT) категорически недопустима.
- **HSTS (HTTP Strict Transport Security):** Заголовок `Strict-Transport-Security: max-age=31536000; includeSubDomains`, принуждающий браузер выполнять все последующие запросы исключительно по HTTPS, предотвращая атаки SSL-Strip.
- **CORS (Cross-Origin Resource Sharing):** Механизм браузера, регулирующий доступ скриптов с одного домена (Origin: `https://frontend.com`) к ресурсам другого домена (`https://api.backend.com`). Включает preflight-запросы `OPTIONS`. Настраивается на сервере через `CorsConfigurationSource`.

Глава 2. Понятийный аппарат Spring Security

Spring Security построен на строгом разделении обязанностей между интерфейсами. Чтобы уверенно ориентироваться в коде фреймворка, необходимо четко различать фундаментальные абстракции.



2.1. Основные сущности и их назначение

Интерфейс / Класс	Смысл и ответственность	Методы и свойства
<code>SecurityContextHolder</code>	Статический фасад (точка доступа) к текущему контексту безопасности. По умолчанию использует стратегию <code>ThreadLocal</code> (один контекст на поток выполнения HTTP-запроса).	<code>getContext()</code> , <code>setContext(context)</code> , <code>clearContext()</code> .
<code>SecurityContext</code>	Контейнер, связывающий текущий поток выполнения с объектом <code>Authentication</code> .	<code>getAuthentication()</code> , <code>setAuthentication(auth)</code> .
<code>Authentication</code>	Двуликый объект: до аутентификации содержит сырые реквизиты (логин/пароль), после — подтвержденного пользователя с правами.	<code>getPrincipal()</code> , <code>getCredentials()</code> , <code>getAuthorities()</code> , <code>isAuthenticated()</code> .
<code>Principal</code>	Объект, идентифицирующий личность. Обычно это экземпляр <code>UserDetails</code> или кастомный класс домена.	В простейшем случае — строка (username), в продвинутом — объект доменной сущности.
<code>GrantedAuthority</code>	Неделимое право (полномочие), выданное пользователю (например, <code>READ_PRIVILEGE</code> или <code>ROLE_ADMIN</code>).	<code>String getAuthority()</code> .
<code>UserDetails</code>	Адаптер между доменной моделью вашей базы данных (таблица <code>users</code>) и требованиями Spring Security.	<code>getUsername()</code> , <code>getPassword()</code> , <code>getAuthorities()</code> , флаги <code>isAccountNonExpired()</code> , <code>isEnabled()</code> .
<code>UserDetailsService</code>	Единственная обязанность — по строковому имени (логину/email) загрузить <code>UserDetails</code> из источника данных (БД, LDAP, API).	<code>UserDetails</code> <code>loadUserByUsername(String username)</code> .
<code>PasswordEncoder</code>	Криптографический компонент для одностороннего хеширования и безопасного сравнения паролей с солью (salt).	<code>String encode(CharSequence rawPassword)</code> , <code>boolean matches(raw, encoded)</code> .

2.2. Разница между Role и Authority

В Spring Security оба понятия представляются через интерфейс `GrantedAuthority` , однако имеют разную семантику:

- **Authority (Полномочие / Право):** Гранулярное действие над объектом системы. Примеры: `PRODUCT_CREATE` , `USER_DELETE` , `REPORT_EXPORT` . Проверяется выражением `hasAuthority('PRODUCT_CREATE')` .
- **Role (Роль):** Агрегат прав, бизнес-статус субъекта. Примеры: `ADMIN` , `MANAGER` . Spring Security по умолчанию требует, чтобы строковое представление роли начиналось с префикса `ROLE_` (например, `ROLE_ADMIN`). Метод `hasRole('ADMIN')` автоматически подставляет префикс и ищет authority с именем

2.3. Глубокое погружение: Диспетчеризация и цепочка проверки подлинности

Вся архитектура аутентификации в Spring Security построена по классическому поведенческому паттерну «Цепочка обязанностей» (Chain of Responsibility) в сочетании с «Медиатором / Диспетчером».

1. AuthenticationManager — Директор бюро пропусков

Интерфейс `AuthenticationManager` не занимается непосредственной проверкой паролей, обращением к базе данных или парсингом токенов. Его зона ответственности — **оркестрация и координация**. Он получает на вход неаутентифицированный объект `Authentication` (например, `UsernamePasswordAuthenticationToken` с логином и паролем) и опрашивает зарегистрированных провайдеров.

```
package org.springframework.security.authentication;

public interface AuthenticationManager {
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
}
```

2. ProviderManager — Стандартная реализация менеджера

Основной реализацией `AuthenticationManager` по умолчанию является класс `ProviderManager`. Внутри него хранится список провайдеров `List<AuthenticationProvider>`. При вызове `authenticate()` он выполняет следующий алгоритм:

1. В цикле опрашивает каждый `AuthenticationProvider` через метод `provider.supports(authentication.getClass())`.
2. Если провайдер возвращает `true` (сообщает: «Да, я умею проверять данный тип учетных данных»), управление передается его методу `provider.authenticate(authentication)`.
3. Если аутентификация успешна, возвращается полностью заполненный объект `Authentication` (со свойством `isAuthenticated() == true`, заполненным `Principal` и правами `Authorities`, а пароль `Credentials` затирается).
4. Если провайдер выбросил `AuthenticationException` (например, `BadCredentialsException`), процесс прерывается, и ошибка пробрасывается вверх.

3. Виды AuthenticationProvider

Провайдер	Специализация	Источник валидации
DaoAuthenticationProvider	Классический вход по логину/паролю из БД.	Использует связку UserDetailsService (поиск в БД) + PasswordEncoder (сверка хэша).
LdapAuthenticationProvider	Корпоративная учетная запись Active Directory / LDAP.	LDAP-сервер предприятия.
JwtAuthenticationProvider	Проверка цифровой подписи и claims токена JWT.	Публичный ключ или HMAC-секрет.
AnonymousAuthenticationProvider	Присваивает анонимный контекст неавторизованным гостям.	Константа ROLE_ANONYMOUS.

4. Анатомия работы DaoAuthenticationProvider под капотом

Когда пользователь отправляет запрос на вход (например, `POST /login/auth`), `DaoAuthenticationProvider` выполняет 5 строгих шагов:

- Запрос к архиву:** Вызывает `userDetailsService.loadUserByUsername(username)`. Сервис обращается через JPA к базе данных и возвращает объект `UserDetails` (в нашем проекте это `AppPrincipal`). Если пользователь не найден — выбрасывается `UsernameNotFoundException`.
- Проверка статусов учетной записи:** Провайдер вызывает методы `isAccountNonLocked()`, `isEnabled()`, `isAccountNonExpired()`. Если хотя бы один флаг равен `false`, выбрасывается специфическое исключение (например, `DisabledException` или `LockedException`).
- Проверка пароля:** Провайдер берет сырой пароль, присланный клиентом, и хэш из `UserDetails.getPassword()`, после чего вызывает `passwordEncoder.matches(rawPassword, encodedPassword)`. Если хэши не совпали — выбрасывается `BadCredentialsException`.
- Очистка учетных данных (Credentials Erasing):** В целях безопасности пароль удаляется из оперативной памяти, чтобы он случайно не утек в логи или дампы кучи (Heap Dump).
- Создание подтвержденного токена:** Создается новый экземпляр `UsernamePasswordAuthenticationToken`, содержащий `Principal` (объект `AppPrincipal`), `Credentials = null` и коллекцию подтвержденных прав `Authorities`.

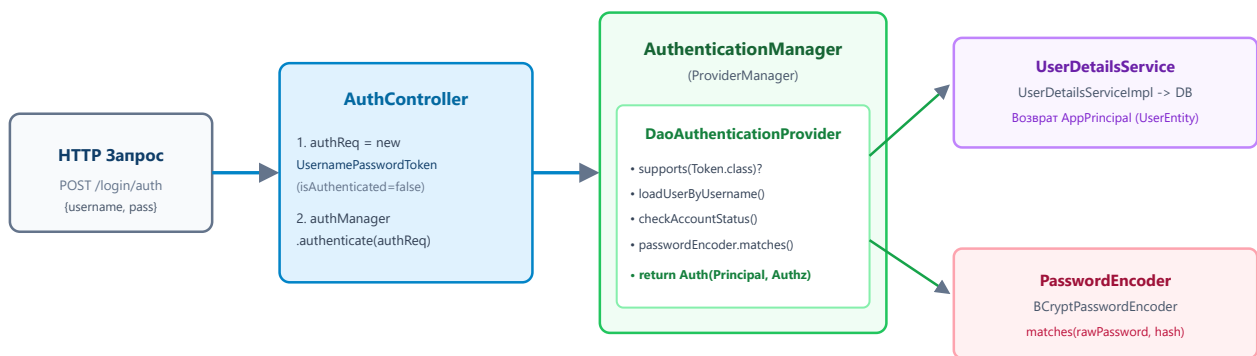
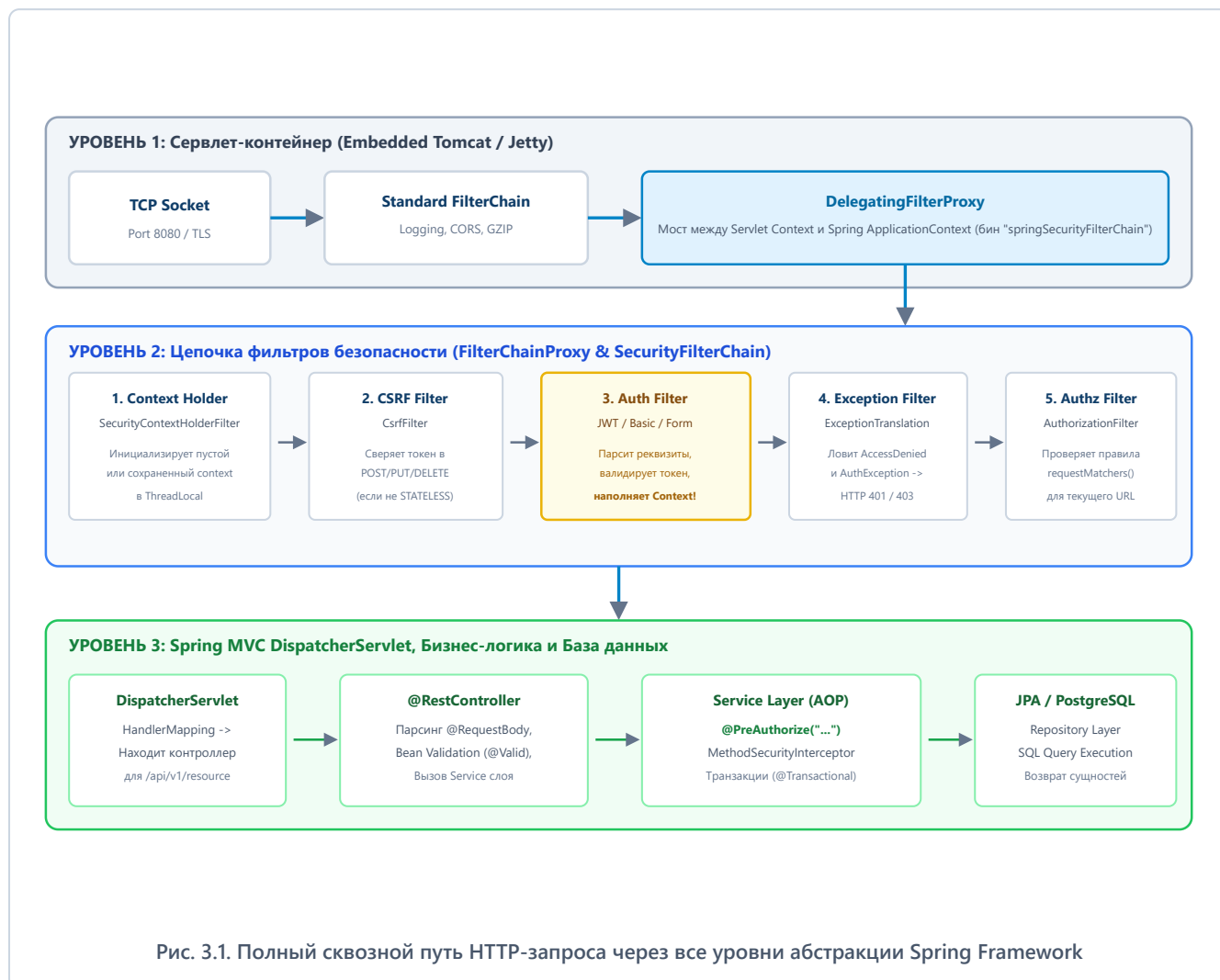


Рис. 2.2. Сквозная цепочка вызовов при обработке логина через AuthenticationManager и DaoAuthenticationProvider

Глава 3. Архитектура под капотом: от сокета до контроллера

Для глубокого понимания Spring Security необходимо проследить точный путь входящего байтового потока от сетевого сокета операционной системы до вызова бизнес-логики в Spring-бине.



3.1. Детальный разбор каждого шага потока обработки

- Сетевой транспорт (Tomcat Connector):** Запрос десериализуется из бинарного потока сокета в стандартные Java-объекты `HttpServletRequest` и `HttpServletResponse`.
- DelegatingFilterProxy:** Стандартный класс сервлетов из пакета `org.springframework.web.filter`. Он регистрируется в контейнере сервлетов и перенаправляет вызовы методу `doFilter()` Spring-бина с именем `springSecurityFilterChain`. Это позволяет использовать внутри фильтра внедрение зависимостей (Dependency Injection).
- FilterChainProxy:** Сердце Spring Security. Содержит список цепочек `SecurityFilterChain`. На основе URL-паттерна он выбирает подходящую цепочку фильтров для текущего запроса.
- SecurityContextHolderFilter:** Первый фильтр безопасности. Он извлекает существующий `SecurityContext` из хранилища (например, из HTTP-сессии, если включен stateful режим) и помещает его в `SecurityContextHolder` текущего потока. В конце запроса он гарантированно очищает

`ThreadLocal`, предотвращая утечку данных в пул потоков Tomcat.

5. **Аутентификационный фильтр (JWT / Basic / Form):** Анализирует заголовки (`Authorization`) или тело. Если реквизиты корректны, конструирует объект `Authentication` и помещает его в `SecurityContextHolder.getContext().setAuthentication(...)`.
6. **ExceptionTranslationFilter:** Перехватывает исключения безопасности. Если выброшен `AuthenticationException` (пользователь анонимен), фильтр запускает `AuthenticationEntryPoint` (отдает 401 или редиректит на форму входа). Если выброшен `AccessDeniedException` (пользователь вошел, но прав недостаточно), запускается `AccessDeniedHandler` (отдает HTTP 403 Forbidden).
7. **AuthorizationFilter (бывший FilterSecurityInterceptor):** Выполняет авторизацию на уровне URL (проверяет правила, заданные в `authorizeHttpRequests`). Если доступ запрещен, бросает `AccessDeniedException` .
8. **DispatcherServlet и контроллеры:** Запрос попадает в стандартный MVC-конвейер, где выполняется привязка параметров, валидация DTO через Hibernate Validator (`@Valid`) и вызов бизнес-методов.
9. **Method Security (AOP):** Если на сервисе объявлена аннотация `@PreAuthorize("hasAuthority('...')")` , Spring CGLIB-прокси перехватывает вызов метода до его выполнения, вычисляет SpEL-выражение над текущим `SecurityContext` и разрешает или блокирует выполнение.

Глава 4. Факторы и методы аутентификации

Аутентификация строится на предоставлении субъектом одного или нескольких независимых доказательств его подлинности (факторов).

4.1. Классификация факторов аутентификации

Фактор	Категория	Примеры реализации	Степень уязвимости
Knowledge (Знание)	«То, что субъект знает»	Пароль, кодовое слово, графический ключ, PIN-код.	Высокая (подвержен фишингу, перебору, утечкам баз данных).
Possession (Владение)	«То, чем субъект владеет»	Смартфон (SMS/Push), TOTP-генератор (Google Authenticator), криптографический USB-токен (YubiKey), смарт-карта.	Средняя/Низкая (требует физического или программного завладения устройством).
Inherence (Свойство)	«То, чем субъект является»	Отпечаток пальца, сканер сетчатки/радужки глаза, 3D Face ID, голосовой слепок.	Крайне низкая (но невозможно скомпрометированный биометрический слепок «сменить», как пароль).

4.2. Механизмы двухфакторной аутентификации (2FA / MFA)

Двухфакторная аутентификация (2FA) строго требует предъявления доказательств из двух *разных* категорий (например, Пароль [Knowledge] + TOTP код [Possession]). Запрос двух паролей подряд не является 2FA.

1. SMS и E-mail одноразовые пароли (OTP)

Сервер генерирует случайное число (обычно 6 цифр), сохраняет его в кэше с TTL (например, 120 секунд) и отправляет провайдеру связи.

Недостатки: Уязвимость к атакам перехвата SS7 в сотовых сетях, перевыпуск SIM-карты злоумышленником (SIM swap), задержки доставки операторами.

2. Алгоритм TOTP (Time-based One-Time Password, RFC 6238) — Google Authenticator

Ключевое преимущество TOTP — генератор работает **полностью автономно** без подключения к интернету и сотовой сети.

- **Фаза настройки:** Сервер генерирует криптостойкий общий секретный ключ (Shared Secret, \$K\$) в формате Base32 (например, JBSWY3DPENPK3RXR). Пользователю отображается QR-код с URI схемы `otpauth://totp/App:user@mail.com?secret=JBSWY3DPENPK3RXR&issuer=App`. Приложение сканирует QR-код и сохраняет ключ \$K\$.
- **Фаза генерации:** Текущее время Unix (в секундах) делится на временной шаг (обычно $T_0 = 30$ сек):
$$T = \lfloor \text{currentTime} / 30 \rfloor$$

Вычисляется HMAC-хэш: $\text{HMAC-SHA1}(K, T)$. Из хэша по алгоритму динамического усечения (Dynamic Truncation) извлекаются 6 десятичных цифр.
- **Фаза проверки:** Сервер выполняет аналогичное математическое вычисление для текущего T , а также для $T-1$ и $T+1$ (допуск на рассинхронизацию часов на клиенте). Если значения совпадают — аутентификация успешна.

3. Аппаратные ключи FIDO2 / WebAuthn

Современный золотой стандарт безопасности. Базируется на асимметричной криптографии (открытый/закрытый ключ). Закрытый ключ генерируется внутри защищенного чипа аппаратного токена и никогда не покидает его. Защищает от фишинга на 100%, так как браузер передает токenu криптографическую привязку к конкретному доменному имени (Origin).

Глава 5. Базовая настройка Spring Security с нуля

В современных версиях Spring Security (начиная с версий 5.7 и 6.x) устарел конфигурационный класс `WebSecurityConfigurerAdapter`. Конфигурация строится на декларативном объявлении бинов компонентов.

5.1. Подключение зависимостей (Gradle / Maven)

```
// build.gradle.kts
dependencies {
    implementation("org.springframework.boot:spring-boot-starter-web")
    implementation("org.springframework.boot:spring-boot-starter-security")
    implementation("org.springframework.boot:spring-boot-starter-validation")
    testImplementation("org.springframework.security:spring-security-test")
}
```

5.2. Поведение «из коробки» (Default Behavior)

Сразу после подключения `spring-boot-starter-security` без написания единой строки кода активируются дефолтные автоконфигурации (`SecurityAutoConfiguration`):

- Все HTTP-эндпоинты приложения закрываются правилом `authenticated()`.
- Генерируется пользователь по умолчанию с логином `user` и случайным UUID-паролем, который выводится в консоль при старте: `Using generated security password: ...`.
- Активируется базовая форма входа (`DefaultLoginPageGeneratingFilter`) по адресу `/login`.
- Включается CSRF-защита и поддержка HTTP-сессий (Stateful).

5.3. Эталонная конфигурация SecurityConfiguration (Spring Security 6.x)

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfiguration {

    @Bean
    public PasswordEncoder passwordEncoder() {
        // BCrypt с фактором трудоемкости 12 (4096 раундов хэширования)
        return new BCryptPasswordEncoder(12);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            // 1. Отключаем CSRF (для REST API со stateless токенами)
            .csrf(AbstractHttpConfigurer::disable)
            // 2. Отключаем сессии на сервере (Stateless архитектура)
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            // 3. Настройка прав доступа к URL
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**", "/public/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/products/**").hasAuthority("PRODUCT_READ")
                .requestMatchers("/api/v1/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            // 4. Обработка ошибок аутентификации и доступа
            .exceptionHandling(exceptions -> exceptions
                .authenticationEntryPoint((request, response, authException) ->
                    response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Необходима аутентификация")
                )
                .accessDeniedHandler((request, response, accessDeniedException) ->
                    response.sendError(HttpServletResponse.SC_FORBIDDEN, "Недостаточно прав доступа")
                )
            )
            .build();
    }

    @Bean
    public AuthenticationManager authenticationManager(
        AuthenticationConfiguration authenticationConfiguration) throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }
}
```

5.4. Построчный анатомический разбор конфигурации

Строка / Фрагмент кода	Техническое назначение и влияние на поведение системы
<code>@EnableWebSecurity</code>	Включает интеграцию Spring Security с Spring MVC, регистрирует бин <code>springSecurityFilterChain</code> .
<code>@EnableMethodSecurity</code>	Активирует AOP-перехватчики для аннотаций <code>@PreAuthorize</code> , <code>@PostAuthorize</code> , <code>@Secured</code> над методами бинов.
<code>new BCryptPasswordEncoder(12)</code>	Генерирует криптографическую соль (16 байт) и выполняет $2^{12} = 4096$ итераций хэширования, защищая от перебора на GPU/ASIC.
<code>.sessionCreationPolicy(STATELESS)</code>	Приказывает Spring Security никогда не создавать <code>HttpSession</code> и не использовать ее для получения <code>SecurityContext</code> .
<code>.permitAll()</code> vs <code>.authenticated()</code>	<code>permitAll()</code> пропускает любые запросы к указанному матчеру без проверки; <code>authenticated()</code> требует валидного <code>Authentication</code> в контексте.

5.5. Секреты объявления бина `AuthenticationManager`

Один из самых частых вопросов разработчиков: «Зачем мы пишем метод `@Bean public AuthenticationManager ...` если Spring Security и так уже работает?».

1. Проблема скрытого менеджера (`NoSuchBeanDefinitionException`)

По умолчанию Spring Security создает экземпляр `AuthenticationManager` глубоко внутри собственных сервлет-фильтров и **не экспортирует его в `ApplicationContext`** в виде публичного бина. Если попытаться внедрить менеджер в контроллер напрямую без явного объявления метода `@Bean`:

```
@RestController
@RequiredArgsConstructor
public class AuthController {
    // ПРИВЕДЕТ К ОШИБКЕ NoSuchBeanDefinitionException при старте приложения!
    private final AuthenticationManager authenticationManager;
}
```

2. Два канонических способа объявления бина

Способ	Код реализации	Особенности применения
Способ 1 (Рекомендуемый): Через <code>AuthenticationConfiguration</code>	<pre>@Bean public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig) throws Exception { return authConfig.getAuthenticationManager(); }</pre>	Стандарт Spring Boot 3 / Security 6. Автоматически находит зарегистрированные в контексте бины <code>UserDetailsService</code> и <code>PasswordEncoder</code> , собирает под капотом <code>DaoAuthenticationProvider</code> и выставляет менеджер наружу.
Способ 2 (Явный): Через ручное создание <code>ProviderManager</code>	<pre>@Bean public AuthenticationManager authenticationManager(UserDetailsService userDetailsService, PasswordEncoder passwordEncoder) { DaoAuthenticationProvider provider = new DaoAuthenticationProvider(); provider.setUserDetailsService(userDetailsService); provider.setPasswordEncoder(passwordEncoder); return new ProviderManager(provider); }</pre>	Используется, когда требуется подключить несколько провайдеров одновременно (например, <code>DaoAuthenticationProvider</code> + <code>LdapAuthenticationProvider</code>) или задать нестандартные параметры кэширования пользователей.

3. Что скрывается внутри AuthenticationConfiguration?

При обращении к исходникам класса `AuthenticationConfiguration` в IDE можно увидеть внутренние конфигураторы:

- `InitializeUserDetailsServiceBeanManagerConfigurer` — служебный класс, сканирующий Spring-контекст на наличие бина `UserDetailsService`.
- `InitializeAuthenticationProviderBeanManagerConfigurer` — конфигуратор, связывающий найденный `UserDetailsService` и `PasswordEncoder` в стандартный `DaoAuthenticationProvider`.
- Метод `getAuthenticationManager()` строит и возвращает готовый синглтон `ProviderManager`.

Глава 6. HTTP Basic Authentication

6.1. Спецификация протокола (RFC 7617)

HTTP Basic — старейший стандартизированный метод аутентификации. Клиент передает учетные данные в каждом запросе в HTTP-заголовке `Authorization`.

```
Authorization: Basic YWRtaW46cGFzc3dvcmQxMjM=
```

Значение после префикса `Basic` формируется путем конкатенации логина, двоеточия и пароля, закодированных по алгоритму Base64:

```
"admin:password123" --[Base64.encode]--> "YWRtaW46cGFzc3dvcmQxMjM="
```

Критическая ошибка понимания: Base64 — это не шифрование!

Base64 — это алгоритм обратимого позиционного кодирования бинарных данных в ASCII-символы. Любой человек или узел сети, перехвативший заголовок, декодирует пароль за 1 миллисекунду. Использование Basic Auth без TLS/HTTPS абсолютно недопустимо.

6.2. Пошаговая реализация Basic Auth в коде

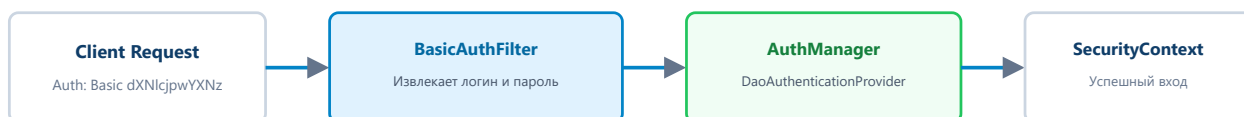


Рис. 6.1. Жизненный цикл запроса Basic Authentication

```

// 1. Доменная сущность UserDetails
public class CustomUserDetails implements UserDetails {
    private final UserEntity user;

    public CustomUserDetails(UserEntity user) { this.user = user; }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority(role.getName()))
            .collect(Collectors.toSet());
    }
    @Override public String getPassword() { return user.getPasswordHash(); }
    @Override public String getUsername() { return user.getUsername(); }
    @Override public boolean isEnabled() { return user.isActive(); }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isAccountNonLocked() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
}

// 2. Сервис загрузки из БД
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {
    private final UserRepository userRepository;

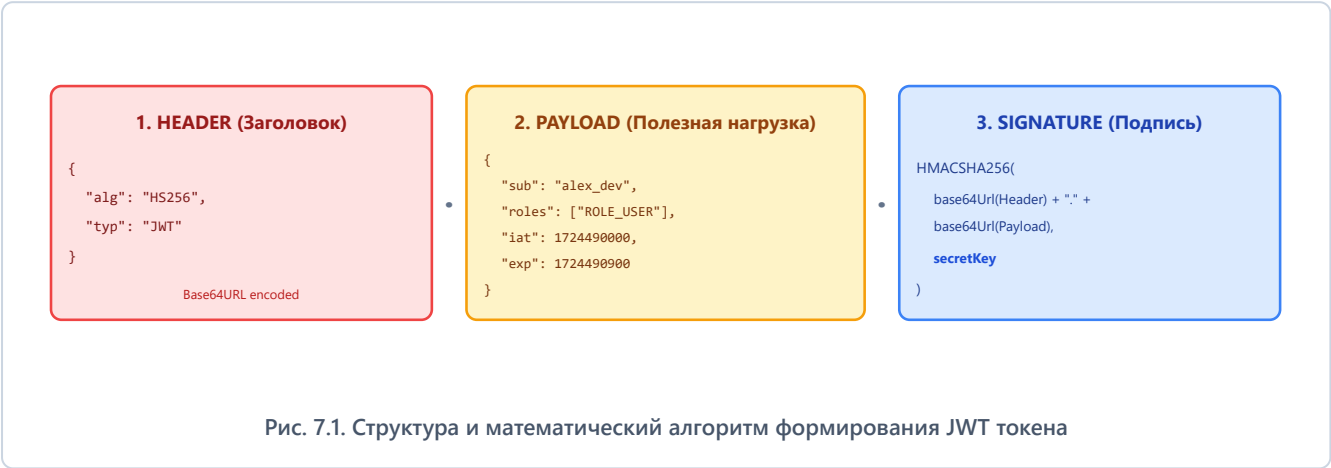
    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        UserEntity user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("Пользователь не найден: " + username));
        return new CustomUserDetails(user);
    }
}

```

Глава 7. Архитектура и реализация JWT (JSON Web Tokens)

7.1. Спецификация структуры токена (RFC 7519)

JWT представляет собой строку из трех сегментов, разделенных точками: `HEADER.PAYLOAD.SIGNATURE` .



7.2. Разница между JWS (Signing) и JWE (Encryption)

- **JWS (JSON Web Signature):** Данные внутри Payload *открыты для чтения* всем участникам обмена. Подпись гарантирует только то, что данные не были подделаны. Подходит для токенов авторизации API.
- **JWE (JSON Web Encryption):** Полезная нагрузка зашифрована симметричным или асимметричным шифром (AES-GCM, RSA-OAEP). Содержимое Payload невозможно прочитать без ключа дешифрования. Используется для передачи конфиденциальных PII-данных.

7.3. Симметричная (HMAC) vs Асимметричная подпись (RSA / ECDSA)

Критерий	Симметричная (HS256, HS384, HS512)	Асимметричная (RS256, ES256, EdDSA)
Ключи	Один общий секрет (Shared Secret).	Пара: Private Key (закрытый) и Public Key (открытый).
Генерация подписи	Секретным ключом на сервере авторизации.	Private Key на Authorization Server.
Проверка подписи	Тем же секретным ключом.	Public Key (можно раздавать сотням микросервисов).
Сценарий применения	Монолитные приложения или единый бэкэнд.	Микросервисная архитектура, OAuth 2.0 / OpenID Connect.

7.4. Эталонный класс JwtService (на базе библиотеки JJWT 0.12+)

```
@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secretKey;

    @Value("${app.jwt.access-token-expiration}")
    private Duration tokenExpiration;

    public String generateToken(UserDetails userDetails) {
        Instant now = Instant.now();
        List<String> roles = userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .toList();

        return Jwts.builder()
            .subject(userDetails.getUsername())
            .claim("roles", roles)
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plus(tokenExpiration)))
            .signWith(getSigningKey(), Jwts.SIG.HS256)
            .compact();
    }

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    public <T> T extractClaim(String token, Function<Claims, T> claimsResolver) {
        final Claims claims = parseAllClaims(token);
        return claimsResolver.apply(claims);
    }

    private Claims parseAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

7.5. Аутентификационный фильтр JwtAuthenticationFilter

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(
        @NonNull HttpServletRequest request,
        @NonNull HttpServletResponse response,
        @NonNull FilterChain filterChain) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");
        final String bearerPrefix = "Bearer ";

        // Если заголовок отсутствует или не начинается с Bearer - пропускаем дальше по цепочке
        if (authHeader == null || !authHeader.startsWith(bearerPrefix)) {
            filterChain.doFilter(request, response);
            return;
        }

        final String jwt = authHeader.substring(bearerPrefix.length()).trim();
        try {
            final String username = jwtService.extractUsername(jwt);

            // Проверяем, что username извлечен и пользователь еще не аутентифицирован в контексте
            if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);

                if (jwtService.isTokenValid(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken
                    (
                        userDetails,
                        null, // Пароль обнуляется в целях безопасности
                        userDetails.getAuthorities()
                    );
                    authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

                    // Наполняем SecurityContext!
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception e) {
            // Ошибки валидации токена (истек, поврежден) приводят к очистке контекста
            SecurityContextHolder.clearContext();
        }

        filterChain.doFilter(request, response);
    }
}
```

Глава 8. Двухтокеновая схема: JWT Access + Refresh Token

8.1. Фундаментальная проблема одиночного Access токена

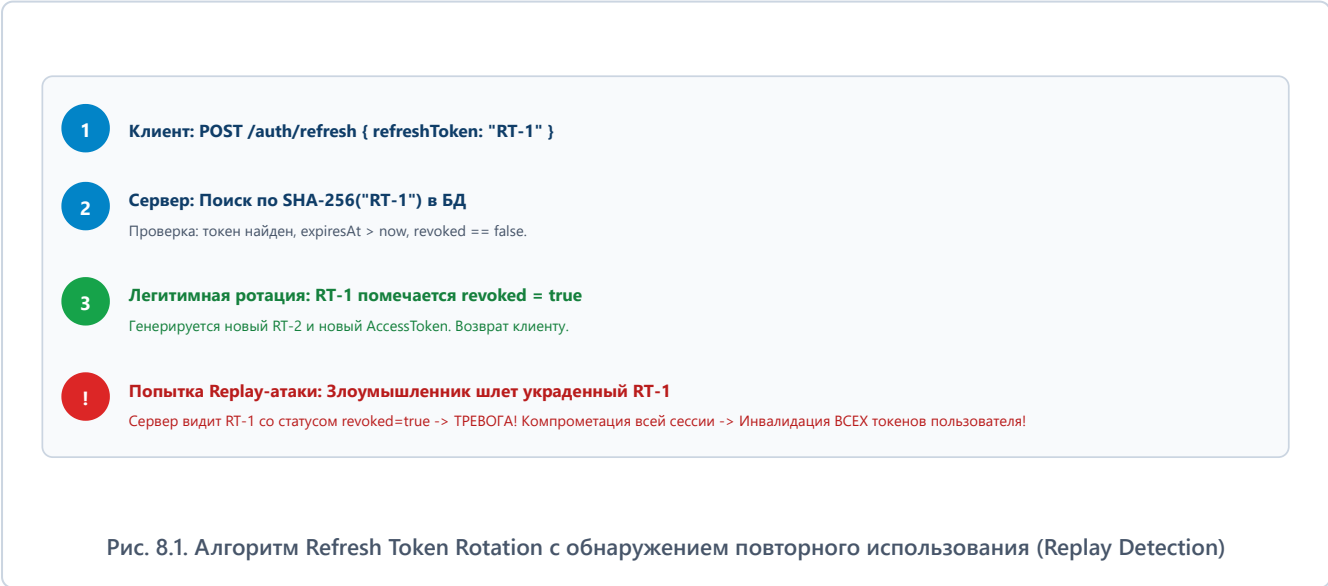
Поскольку JWT является самодостаточным (stateless) токеном, сервер проверяет только его математическую подпись и срок действия `exp`. Сервер **не может отозвать** валидный Access Token до истечения его срока действия без введения централизованного черного списка (Blacklist в Redis), что уничтожает идею stateless.

- Если сделать Access Token долгоживущим (например, 30 дней) — в случае кражи токена злоумышленник владеет аккаунтом целый месяц.
- Если сделать Access Token короткоживущим (10 минут) — пользователю придется каждые 10 минут заново вводить логин и пароль.

8.2. Решение: разделение ролей Access и Refresh токенов

Характеристика	Access Token (Токен доступа)	Refresh Token (Токен обновления)
Формат	Подписанный JWT (JWS).	Непрозрачная случайная криптостойкая строка (UUID или SecureRandom 64 байта).
Время жизни (TTL)	Короткое: от 5 до 15 минут.	Длинное: от 7 до 60 дней.
Хранение на бэкенде	Нигде не сохраняется (Stateless).	Обязательно сохраняется в БД / Redis (только в виде SHA-256 хэша!).
Область применения	Отправляется в каждом запросе к бизнес-API: <code>Authorization: Bearer ...</code> .	Отправляется только на эндпоинт <code>POST /api/v1/auth/refresh</code> .

8.3. Ротация Refresh-токенов (Token Rotation) и защита от Replay-атак



8.4. Эталонный сервис RefreshTokenService

```
@Service
@RequiredArgsConstructor
public class RefreshTokenService {

    @Value("${app.jwt.refresh-token-expiration}")
    private Duration refreshExpiration;

    private final RefreshTokenRepository refreshTokenRepository;
    private final UserRepository userRepository;
    private final JwtService jwtService;

    @Transactional
    public String createRefreshToken(String username) {
        UserEntity user = userRepository.findByUsername(username)
            .orElseThrow(() -> new EntityNotFoundException("User not found"));

        String rawToken = UUID.randomUUID().toString();
        String tokenHash = hashToken(rawToken);

        RefreshTokenEntity entity = RefreshTokenEntity.builder()
            .user(user)
            .tokenHash(tokenHash)
            .expiresAt(Instant.now().plus(refreshExpiration))
            .revoked(false)
            .build();

        refreshTokenRepository.save(entity);
        return rawToken; // Клиенту отдаем ТОЛЬКО сырой UUID
    }

    @Transactional
    public TokenResponse rotateRefreshToken(String rawRefreshToken) {
        String incomingHash = hashToken(rawRefreshToken);

        RefreshTokenEntity tokenEntity = refreshTokenRepository.findByTokenHash(incomingHash)
            .orElseThrow(() -> new BadCredentialsException("Invalid refresh token"));

        // ОБНАРУЖЕНИЕ АТАКИ ПОВТОРНОГО ИСПОЛЬЗОВАНИЯ (Replay Attack)
        if (tokenEntity.isRevoked()) {
            // Токен уже был использован ранее! Инвалидируем ВСЕ токены пользователя
            refreshTokenRepository.revokeAllUserTokens(tokenEntity.getUser().getId());
            throw new BadCredentialsException("Compromised token reuse detected");
        }

        // Проверка срока действия
        if (tokenEntity.getExpiresAt().isBefore(Instant.now())) {
            tokenEntity.setRevoked(true);
            refreshTokenRepository.save(tokenEntity);
            throw new BadCredentialsException("Refresh token expired");
        }

        // 1. Отзываем текущий токен
        tokenEntity.setRevoked(true);
        refreshTokenRepository.save(tokenEntity);

        // 2. Выпускаем новую пару
        String newRawRefreshToken = createRefreshToken(tokenEntity.getUser().getUsername());
        String newAccessToken = jwtService.generateToken(tokenEntity.getUser());

        return new TokenResponse(newAccessToken, newRawRefreshToken);
    }
}
```

```
private String hashToken(String rawToken) {  
    try {  
        MessageDigest digest = MessageDigest.getInstance("SHA-256");  
        byte[] hash = digest.digest(rawToken.getBytes(StandardCharsets.UTF_8));  
        return HexFormat.of().formatHex(hash);  
    } catch (NoSuchAlgorithmException e) {  
        throw new IllegalStateException("SHA-256 algorithm missing", e);  
    }  
}
```

Глава 9. Протокол OAuth 2.0 (RFC 6749, RFC 7636)

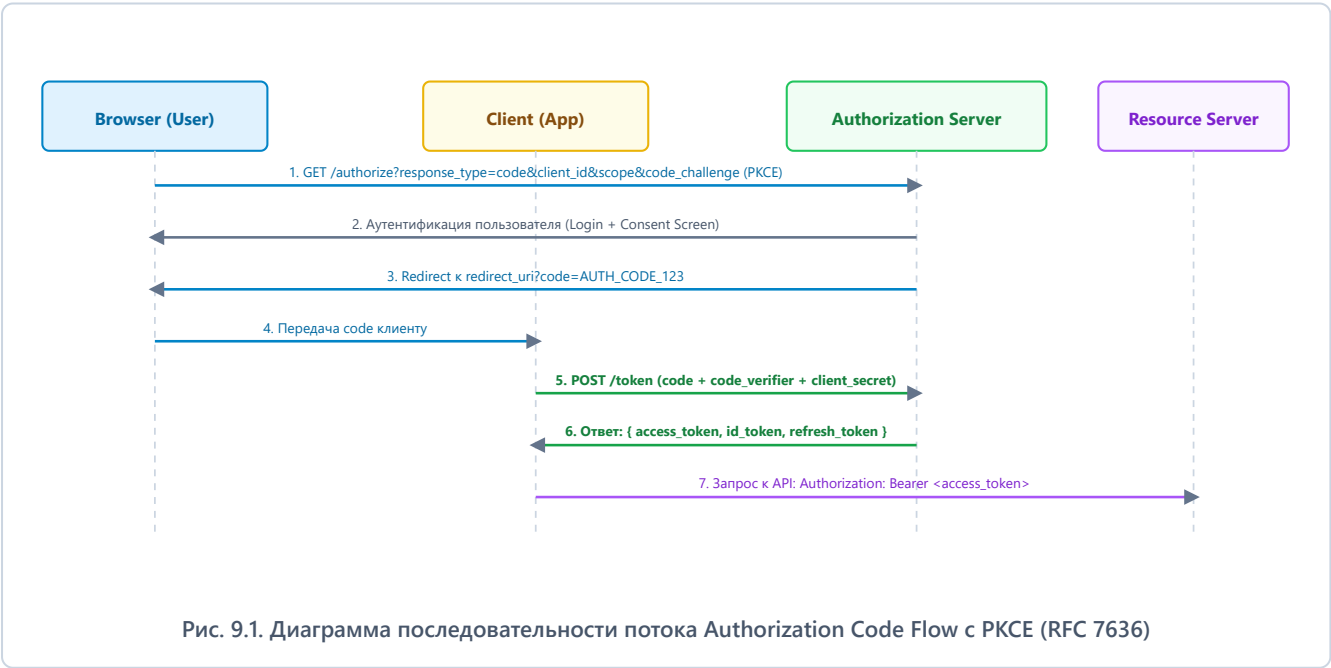
9.1. Концепция делегированной авторизации

OAuth 2.0 — это протокол, решающий фундаментальную задачу: как предоставить стороннему приложению (Client) ограниченный доступ к ресурсам пользователя (Resource Owner) на удаленном сервере (Resource Server) **без передачи этому приложению логина и пароля пользователя**.

9.2. Роли в протоколе OAuth 2.0

Роль	Описание	Пример из реальной жизни
Resource Owner	Владелец данных (пользователь).	Человек с учетной записью Google / GitHub.
Client	Приложение, запрашивающее доступ.	Наш бэкенд или SPA-фронтенд складского учета.
Authorization Server	Сервер, аутентифицирующий владельца и выдающий токены.	Keycloak, Google OAuth Server, Auth0, Okta.
Resource Server	Сервер, хранящий защищенные API и проверяющий токены.	Наш REST API (<code>/api/v1/orders</code> , <code>/api/v1/stock</code>).

9.3. Детальный разбор всех Grant Types (Типов выдачи)



1. Authorization Code Grant + PKCE (Proof Key for Code Exchange, RFC 7636)

Единственный рекомендованный стандарт для пользовательских веб- и мобильных приложений в спецификации OAuth 2.1.

Принцип PKCE: Клиент генерирует случайную строку `code_verifier` и ее SHA-256 хэш `code_challenge`. Сервер авторизации запоминает challenge. При обмене кода на токен клиент присылает исходный `code_verifier`. Это полностью исключает перехват одноразового кода авторизации в браузере.

2. Client Credentials Grant

Используется для взаимодействия **сервис-к-сервису (Machine-to-Machine)** без участия человека. Сервис отправляет свои `client_id` и `client_secret` напрямую на эндпоинт `/token` и получает токен для вызова другого сервиса.

3. Устаревшие и запрещенные в OAuth 2.1 гранты

- **Resource Owner Password Credentials Grant (Password Grant):** Передача логина и пароля пользователя напрямую в клиентское приложение. *Запрещен*, так как уничтожает смысл OAuth.
- **Implicit Grant:** Возврат токена напрямую в URL-фрагменте браузера (`#access_token=...`). *Запрещен* из-за уязвимости к утечкам через историю браузера и заголовок Referer.

Глава 10. OpenID Connect (OIDC Core 1.0)

10.1. Идентификация поверх авторизации

OAuth 2.0 дает только **Access Token** («ключ от двери»). Он ничего не сообщает клиенту о том, *кто* является пользователем.

OpenID Connect (OIDC) — это надстройка над OAuth 2.0, добавляющая концепцию **Идентификации (Identity)**. OIDC вводит стандартизированный токен — **ID Token**.

10.2. Разница между Access Token и ID Token

Критерий	ID Token (OpenID Connect)	Access Token (OAuth 2.0)
Кому адресован (Audience)	Клиентскому приложению (Client). Клиент обязан распарсить и прочитать его.	Ресурсному серверу (Resource Server API). Для клиента токен является непрозрачным.
Формат	Строго стандартизированный JWT.	Спецификацией не определен (может быть JWT или Opaque reference).
Типичные Claims	<code>sub</code> , <code>iss</code> , <code>aud</code> , <code>exp</code> , <code>name</code> , <code>email</code> , <code>email_verified</code> .	<code>scope</code> , <code>client_id</code> , <code>permissions</code> , <code>sub</code> .
Назначение	Отобразить в UI профиль: «Вы вошли как Иван (ivan@mail.com)».	Передать в заголовок <code>Authorization: Bearer</code> для доступа к данным.

10.3. Стандартные эндпоинты OIDC провайдера

- `/.well-known/openid-configuration` (Discovery Document): Метаданные сервера (список URL, поддерживаемые алгоритмы подписи, адрес JWKS).
- `/protocol/openid-connect/certs` (JWKS — JSON Web Key Set): Набор публичных RSA/EC ключей провайдера для проверки подписи токенов ресурсными серверами.
- `/protocol/openid-connect/userinfo` (UserInfo Endpoint): Защищенный API для получения полного профиля пользователя по Access токenu.

Глава 11. Внедрение OAuth 2.0 / OIDC в Spring Boot

11.1. Сценарий 1: Spring Boot как OAuth2 Client (Вход через Keycloak / Google)

```
// 1. Зависимость в build.gradle.kts
implementation("org.springframework.boot:spring-boot-starter-oauth2-client")

// 2. Конфигурация application.yaml
spring:
  security:
    oauth2:
      client:
        registration:
          keycloak:
            client-id: warehouse-client
            client-secret: ${KEYCLOAK_CLIENT_SECRET}
            authorization-grant-type: authorization_code
            scope: openid, profile, email
            redirect-uri: "${baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          keycloak:
            issuer-uri: https://auth.company.com/realms/warehouse-realm

// 3. Конфигурация SecurityFilterChain
@Bean
public SecurityFilterChain oauthClientFilterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/", "/public/**").permitAll()
            .anyRequest().authenticated()
        )
        .oauth2Login(Customizer.withDefaults()) // Автоматический редирект на Keycloak/Google
        .build();
}
```

11.2. Сценарий 2: Spring Boot как Resource Server (Валидация JWT от Keycloak)

```
// 1. Зависимость в build.gradle.kts
implementation("org.springframework.boot:spring-boot-starter-oauth2-resource-server")

// 2. Конфигурация application.yaml
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          // Spring автоматически скачает публичные ключи JWKS с эндпоинта issuer-uri
          issuer-uri: https://auth.company.com/realms/warehouse-realm

// 3. Конфигурация Resource Server
@Configuration
@EnableMethodSecurity
public class ResourceServerConfig {

    @Bean
    public SecurityFilterChain resourceServerFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.GET, "/api/v1/orders/**").hasAuthority("SCOPE_orders.read")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(jwt -> jwt.jwtAuthenticationConverter(customJwtAuthenticationConverter()))
            )
            .build();
    }

    // Конвертер ролей из Keycloak JSON claim "realm_access.roles" в Spring GrantedAuthority
    private Converter<Jwt, AbstractAuthenticationToken> customJwtAuthenticationConverter() {
        JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(jwt -> {
            Map<String, Object> realmAccess = jwt.getClaim("realm_access");
            if (realmAccess == null || !realmAccess.containsKey("roles")) {
                return Collections.emptyList();
            }
            List<String> roles = (List<String>) realmAccess.get("roles");
            return roles.stream()
                .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
                .collect(Collectors.toList());
        });
        return converter;
    }
}
```

Глава 12. Приложение: Промышленная архитектура безопасности

Ниже систематизированы паттерны того, как аутентификация и авторизация строятся в реальных коммерческих проектах в зависимости от масштаба системы.

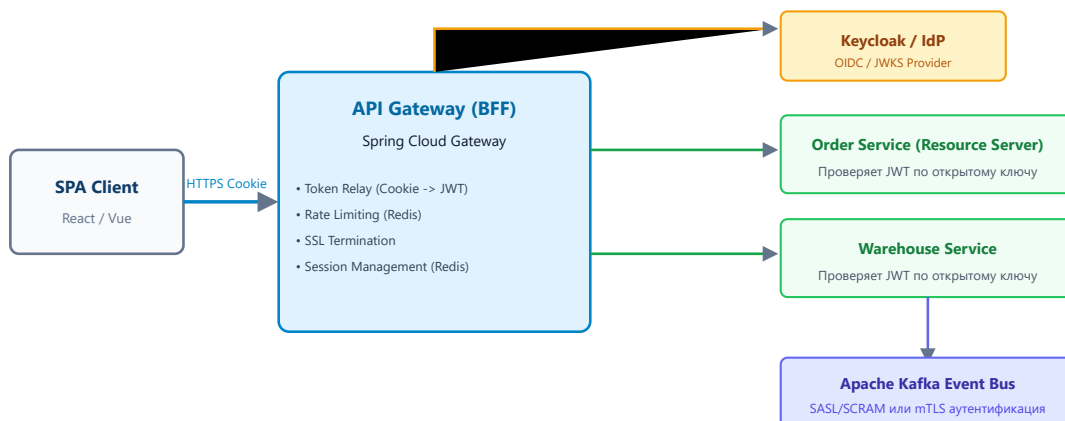


Рис. 12.1. Эталонная архитектура безопасности Enterprise-системы (BFF + Gateway + Keycloak + Resource Servers)

12.1. Сравнение промышленных подходов

Архитектурный паттерн	Стек технологий	Плюсы	Минусы / Сложность
1. Монолит со Spring Session + Redis	Spring Boot, Spring Session Data Redis, PostgreSQL, Thymeleaf / SPA.	Мгновенный отзыв сессии в Redis, простота отладки, нет возни с JWT-ключами.	Привязка к состоянию в Redis, сложность масштабирования на независимые мобильные/ сторонние клиенты.
2. Самописный JWT + Refresh (наш вариант)	Spring Boot, JWT, PostgreSQL / Redis (для Refresh Hash).	Полный контроль над кодом, нулевая внешняя инфраструктура (не нужен Keycloak).	Необходимо вручную поддерживать ротацию, защиту от Replay-атак и код фильтров.
3. Enterprise OAuth 2.0 / OIDC + Keycloak	Keycloak / Auth0, Spring OAuth2 Resource Server, Spring Cloud Gateway.	Единый вход (SSO), управление пользователями через админку, встроенный 2FA/TOTP/WebAuthn, аудит.	Тяжелая инфраструктура (Keycloak требует выделенных ресурсов и отдельного обслуживания).
4. Backend-For-Frontend (BFF Pattern)	SPA (React/Vue) $\rightarrow$ Spring Cloud Gateway (BFF) $\rightarrow$ Микросервисы.	Браузер хранит только безопасный <code>HttpOnly</code> cookie; Gateway меняет cookie на JWT и проксирует его в микросервисы.	Требует выделенного слоя шлюза и настройки Token Relay.

Резюме для инженера

Безопасность — это не отдельная библиотека, а сквозной конвейер. Выбор между Basic Auth, JWT и OAuth 2.0 диктуется исключительно топологией системы. Для компактных сервисов оптимален JWT + Refresh Token; для корпоративных контуров с множеством клиентов и единой авторизацией — OAuth 2.0 / OIDC с выделенным сервером аутентификации (Keycloak).

Глава 13. Руководство по работе с официальной документацией Spring и техническим английским (A2+/B1)

Умение самостоятельно и быстро находить проверенные архитектурные решения в первоисточниках — ключевое отличие профессионального инженера. Официальная документация Spring Framework (портал `docs.spring.io`) исчерпывающая, но требует понимания ее внутренней структуры.

13.1. Структура официального портала `docs.spring.io`

Главная ошибка начинающих — путаница между двумя главными разделами безопасности:

Раздел документации	URL префикс	Вопрос, на который отвечает	Ключевые классы и темы
Authentication (Аутентификация)	<code>/servlet/authentication/...</code>	«Кто ты такой?» (Проверка подлинности субъекта)	<code>AuthenticationManager</code> , <code>AuthenticationProvider</code> , <code>DaoAuthenticationProvider</code> , <code>UserDetailsService</code> , <code>PasswordEncoder</code> , токены, логин/пароль.
Authorization (Авторизация)	<code>/servlet/authorization/...</code>	«Что тебе разрешено делать?» (Контроль доступа)	<code>AuthorizationManager</code> , <code>@PreAuthorize</code> , <code>@Secured</code> , <code>requestMatchers()</code> , роли, права доступа, HTTP 403 Forbidden.

13.2. Типы источников: Javadoc vs Reference Guide vs Guides

- **Reference Documentation (Учебник и книга рецептов):** Адрес `docs.spring.io/spring-security/reference/`. Содержит концептуальные схемы, описание работы фильтров и готовые фрагменты кода для конфигураций. Именно здесь нужно искать ответы на вопросы проектирования.
- **API Javadoc (Синтаксический справочник):** Содержит описания сигнатур отдельных классов, методов, типов параметров и исключений. Полезен, когда класс уже известен, но нужно уточнить назначение конкретного флага.
- **Spring Guides (Пошаговые руководства):** Портал `spring.io/guides` (например, "Securing a Web Application"). Содержит готовые законченные мини-проекты с репозиториями на GitHub.

13.3. Декомпилированный байткод в IDE vs Исходники с комментариями

При попытке заглянуть внутрь классов Spring через IDE (например, `AuthenticationConfiguration.java`) разработчик часто видит «сухой» декомпилированный код без комментариев.

Как включить полноценную документацию прямо в IntelliJ IDEA

1. Нажмите **Ctrl + Q** (на Windows/Linux) или **F1** над любым методом или классом — откроется окно *Quick Documentation* с отформатированным описанием и примерами кода.
2. При открытии стороннего класса нажмите сверху желтой панели кнопку **"Download Sources"** (Скачать исходники). После этого декомпилированный байткод заменится на оригинальные файлы с подробными Javadoc-комментариями от архитекторов Spring.

13.4. Техника эффективного чтения документации на английском (уровень A2+/B1)

Технический английский в документации Spring предельно лаконичен, логичен и не содержит сложного сленга. Для комфортного чтения на уровне A2+/B1 рекомендуется применять 4-шаговый метод:

1. **Шаг 1 (Диаграммы и Код — 70% информации):** Пролистайте страницу до диаграммы вызовов или блока с кодом Java/Kotlin. Названия классов и методов говорят сами за себя.
2. **Шаг 2 (Выделенные плашки Callouts):** Ознакомьтесь с блоками *Tip* (лучшие практики), *Note* (важные нюансы) и *Warning / Deprecated* (устаревшие подходы).
3. **Шаг 3 (Чтение по первым предложениям):** В англоязычной технической литературе главное утверждение всегда выносится в первое предложение абзаца (Topic Sentence). Если первое предложение не относится к вашей задаче — пропускайте весь абзац.
4. **Шаг 4 (Инструменты двуязычного перевода):** Используйте расширения браузера (например, *Immersive Translate*), которые выводят русский перевод под оригинальным английским текстом, оставляя код и технические термины без искажений.

13.5. Микро-словарь ключевых терминов Spring Security

Термин (Английский)	Русский эквивалент	Контекст употребления в Spring Security
delegate	Делегировать / Передавать	<i>Manager delegates authentication to a list of providers.</i>
intercept	Перехватывать	<i>Security filter intercepts incoming HTTP request before MVC dispatcher.</i>
invoke / trigger	Вызывать / Запускать	<i>AuthenticationEntryPoint is invoked when user is unauthenticated.</i>
expose	Экспортировать / Публиковать	<i>Expose AuthenticationManager as a Spring @Bean.</i>
override	Переопределять	<i>Override default security configuration.</i>
bypass	Пропускать / Обходить	<i>Static resources bypass security filter chain.</i>
validate / verify	Проверять валидность	<i>Verify JWT signature using public key.</i>
populate	Заполнять данными	<i>Populate SecurityContext with authenticated Principal.</i>
persist	Сохранять в БД	<i>Persist refresh token hash into PostgreSQL.</i>
extract	Извлекать	<i>Extract username claim from JWT payload.</i>
Principal	Субъект безопасности	Объект текущего аутентифицированного пользователя (<code>UserDetails</code> / <code>AppPrincipal</code>).
Credentials	Реквизиты доступа	Секретные данные для подтверждения личности (пароль, токен, сертификат).
Authority	Полномочие / Право	Неделимое разрешение на действие (<code>UNITS_READ</code> , <code>ROLE_ADMIN</code>).
Prerequisite	Предварительное условие	Требования, которые должны быть выполнены до текущего шага.
Stateless	Без сохранения состояния	Сервер не хранит HTTP-сессии (актуально для JWT REST API).
Downstream	Ниже по конвейеру	Компоненты, вызываемые следующими в цепочке обработки запроса.